# IPROPERTY

## System Architecture Document

ArchiMate 3.1 Compliant Architecture Framework

### Technology Stack

| Component | Technology | Version |
|---|---|---|
| **Frontend** | Flutter / Dart | SDK 3.x |
| **Backend API** | ASP.NET Core | 8.0 |
| **Database** | Oracle Database | 11g Release 2 |
| **Integration** | Oracle Financials | Cloud 24D |
| **Architecture Framework** | ArchiMate | 3.1 |

### Document Information

| Field | Value |
|---|---|
| **Document Title** | IProperty System Architecture Document |
| **Version** | 1.0 |
| **Status** | DRAFT |
| **Classification** | Confidential |
| **Date** | April 2026 |
| **Framework** | ArchiMate 3.1 (Open Group Standard) |

# Document Revision History

The following table records all revisions made to this System Architecture Document.

| Version | Date | Author | Description | Status |
|---|---|---|---|---|
| **0.1** | March 2026 | Architecture Team | Initial Draft | Draft |
| **0.5** | April 2026 | Architecture Team | ArchiMate diagrams added | Review |
| **1.0** | April 2026 | Architecture Team | Final review and approval | Approved |

# 1. Executive Summary

IProperty is a comprehensive real estate property management platform designed to digitalise and streamline the end-to-end lifecycle of property assets, lease agreements, financial operations, and stakeholder interactions. This System Architecture Document (SAD) provides a structured, multi-layered view of the IProperty platform using the ArchiMate 3.1 enterprise architecture modelling notation as defined by The Open Group.

The platform is architected on a modern three-tier stack: a Flutter-based cross-platform mobile front end, an ASP.NET Core 8 RESTful back-end API layer, and an Oracle 11g relational database. A critical integration pillar connects IProperty to Oracle Financials Cloud to ensure seamless financial data exchange across General Ledger (GL), Accounts Payable (AP), Accounts Receivable (AR), Fixed Assets (FA), and Human Resources (HR) modules.

This document covers the Business, Application, Technology, Integration, Data, Security, Deployment, and Operational architecture dimensions and is intended for consumption by enterprise architects, solution architects, developers, database administrators, security officers, and project stakeholders.

### Key Architectural Goals

- Provide a scalable, cloud-ready property management platform using Flutter and .NET Core
- Ensure bi-directional, near-real-time integration with Oracle Financials Cloud
- Maintain enterprise-grade security across all layers (DMZ, Application, Data, Integration)
- Align all architectural decisions to the ArchiMate 3.1 modelling standard
- Support 99.9% uptime through Oracle Data Guard and Azure high-availability patterns

# 2. Architecture Overview

The IProperty architecture follows the ArchiMate 3.1 three-layer model, separating concerns into the Business Layer, the Application Layer, and the Technology Layer. Each layer is connected via well-defined interfaces and relationships, enabling separation of concerns and independent evolution of components.

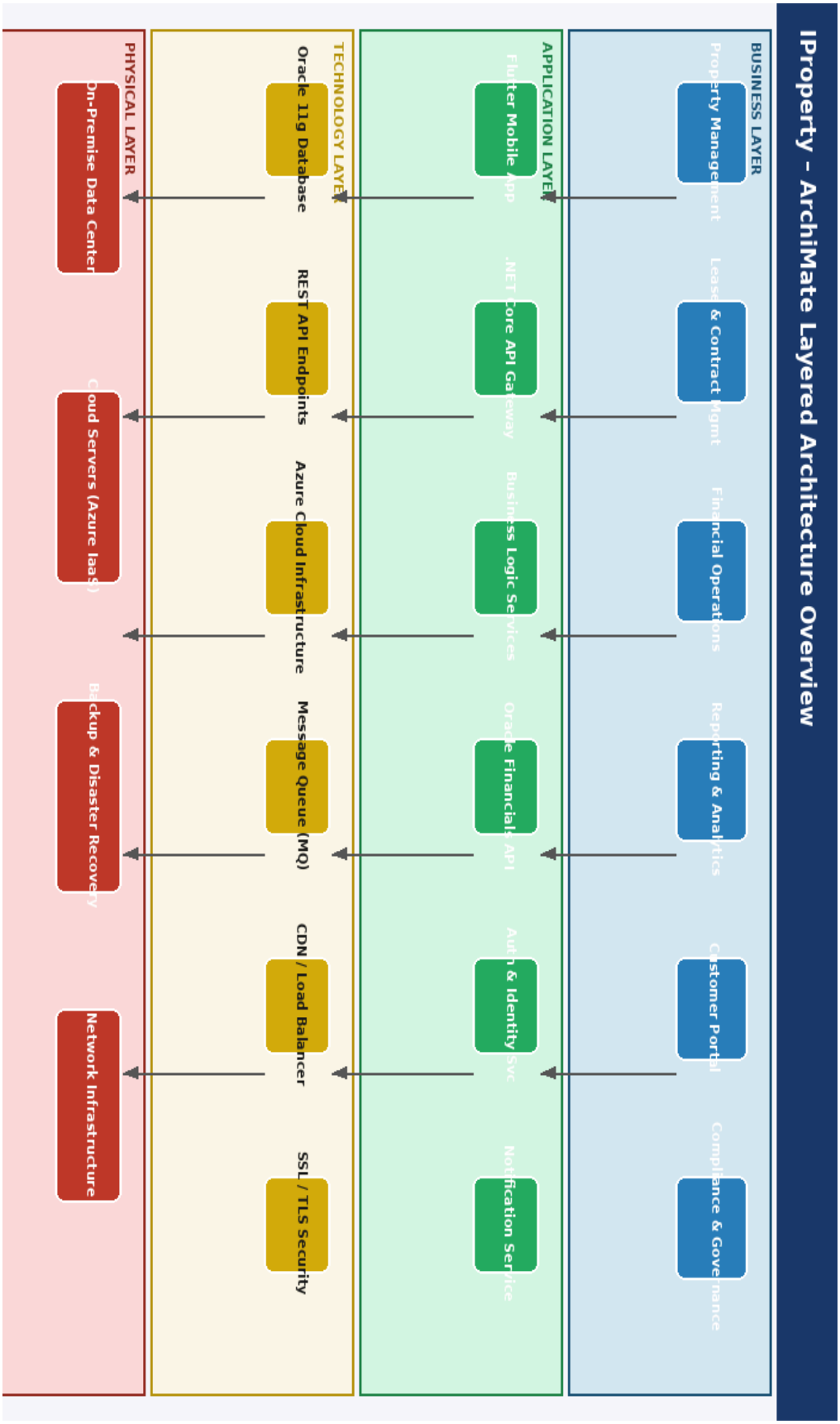


Figure 1 – IProperty ArchiMate Layered Architecture Overview

## 2.1 ArchiMate Layer Summary

ArchiMate is The Open Group's enterprise architecture modelling language. It provides a uniform representation for diagrams and architecture descriptions, and supports the integration of models from different tools and methodologies. The three core layers are described below:

| Layer | ArchiMate Concern | IProperty Realisation |
|---|---|---|
| **Business Layer** | Business actors, roles, processes, functions and services | Property management, lease management, financial operations, reporting, compliance |
| **Application Layer** | Application components, services, data objects and interfaces | Flutter App, .NET Core API Gateway, Business Logic Services, Auth Service, Notifications |
| **Technology Layer** | System software, infrastructure, physical components | Oracle 11g DB, Azure Cloud, REST Endpoints, Message Queue, Load Balancer, SSL/TLS |

## 2.2 ArchiMate Relationship Types Used

The following ArchiMate relationship types are applied throughout this document's architecture models:

| Relationship | Description | Example |
|---|---|---|
| **Realisation** | An application component realises a business service | Flutter App realises the Customer Portal service |
| **Serving** | An element provides services to another | API Gateway serves the Flutter front end |
| **Association** | A general relationship without direction | Oracle DB associated with .NET Core services |
| **Composition** | One element is composed of others | Oracle Financials composed of GL, AP, AR, FA, HR |
| **Flow** | Transfer of information between processes | REST API flows data from .NET Core to Oracle DB |
| **Triggering** | An element triggers another | User action in Flutter triggers an API Gateway call |

# 3. Architecture Principles

Architecture principles serve as the foundation for all design decisions within IProperty. They represent the rules and guidelines that must be upheld across the entire solution lifecycle, from initial design through to operation and decommissioning.

| # | Principle | Statement | Rationale |
|---|---|---|---|
| **P01** | Separation of Concerns | Each layer has a clearly defined responsibility with minimal coupling to other layers | Enables independent scaling, testing, and deployment of front end, back end, and database components |
| **P02** | API-First Design | All system capabilities are exposed through well-defined RESTful APIs | Ensures the Flutter app, Oracle Financials, and third-party systems can consume services uniformly |
| **P03** | Security by Design | Security controls are embedded at every layer, not added as an afterthought | Reduces the attack surface and ensures compliance with data protection regulations |
| **P04** | Cloud-Native Readiness | The application layer is designed for deployment on Azure cloud infrastructure | Enables elastic scalability, managed services, and reduced operational overhead |
| **P05** | Data Integrity | Oracle 11g with Data Guard ensures zero data loss and high availability | Property and financial data is critical; inconsistency has direct business and legal consequences |
| **P06** | Loose Coupling | Services communicate via REST APIs and message queues, not direct database calls | Allows services to evolve independently without cascading failures |
| **P07** | Observability | Structured logging, distributed tracing, and health endpoints are mandatory | Enables proactive issue detection, root cause analysis, and performance optimisation |
| **P08** | ArchiMate Compliance | All architecture documentation follows the ArchiMate 3.1 notation standard | Provides a common, vendor-neutral language for all architecture stakeholders |

# 4. Business Architecture

The Business Architecture layer defines the business actors, roles, processes, and services that IProperty supports. This layer maps to the ArchiMate Business Layer and describes the 'what' and 'who' of the system – the business context in which all technology decisions are made.

## 4.1 Business Actors & Roles

| Actor | Role | Responsibilities |
|---|---|---|
| **Property Manager** | Internal User | Manages property listings, lease agreements, maintenance requests, and tenant communications |
| **Finance Officer** | Internal User | Processes invoices, manages payables/receivables, generates financial reports, and reconciles GL entries |
| **Tenant / Lessee** | External User | Views lease details, raises maintenance requests, pays rent, and accesses the customer portal |
| **Property Owner** | External User | Reviews property performance reports, approves budgets, and monitors occupancy and financials |
| **System Administrator** | Internal User | Manages user access, system configuration, integrations, and security policies |
| **Oracle Financials** | External System | Receives and sends GL, AP, AR, FA, and HR transactions via the integration API layer |

## 4.2 Core Business Processes

The following business processes are supported by IProperty in its initial release scope:

- Property Lifecycle Management – onboarding, activation, modification, and decommissioning of property records
- Lease & Contract Management – creation, approval, renewal, termination, and archiving of lease agreements
- Financial Operations – rent collection, invoice generation, GL posting, AP processing, and AR reconciliation
- Maintenance & Work Order Management – logging, assigning, tracking, and closing maintenance requests
- Reporting & Analytics – occupancy reports, revenue reports, lease expiry dashboards, and financial summaries

- Compliance & Governance – audit trails, document management, regulatory reporting, and access control

## 4.3 Business Capability Map

| Capability Domain | Business Capabilities |
|---|---|
| **Portfolio Management** | Property Registry, Asset Valuation, Occupancy Tracking, Portfolio Reporting |
| **Lease Management** | Lease Creation, Renewal Workflows, Termination Processing, Document Storage |
| **Financial Management** | Billing Engine, Payment Processing, Reconciliation, Budget Management |
| **Maintenance Operations** | Request Intake, Work Order Dispatch, Vendor Management, Completion Tracking |
| **Customer Experience** | Tenant Portal, Self-Service Requests, Notifications, Feedback Management |
| **Compliance & Risk** | Audit Logs, Access Control, Regulatory Reporting, Document Retention |

# 5. Application Architecture

The Application Architecture defines the software components, their responsibilities, and the interfaces through which they interact. IProperty follows a layered, service-oriented application design aligned to ArchiMate Application Layer concepts. The architecture consists of a Flutter-based front end, an ASP.NET Core 8 API gateway and business service layer, and a set of supporting application services.

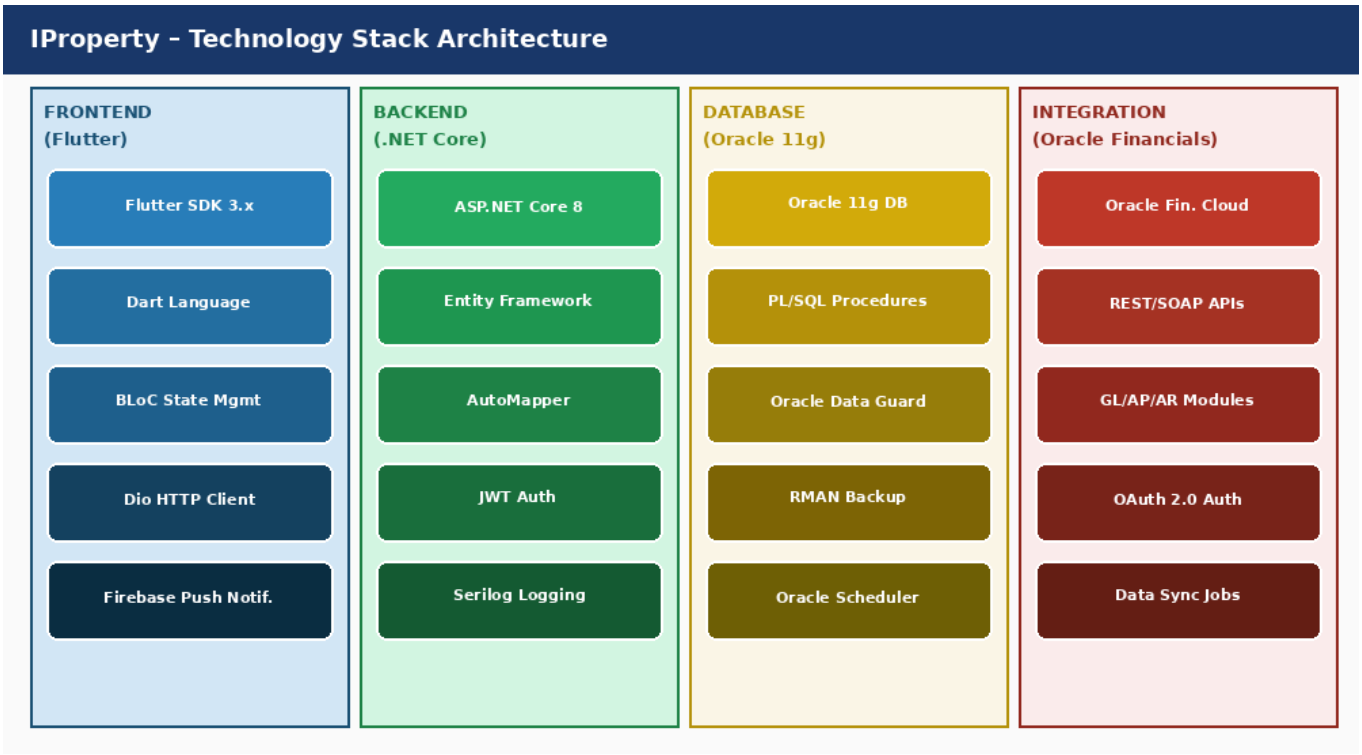


Figure 2 – IProperty Technology Stack Architecture

## 5.1 Flutter Front End (Mobile Application)

The mobile front end is built using the Flutter SDK (v3.x) with Dart as the programming language. Flutter's cross-platform capabilities enable deployment to both Android and iOS from a single codebase, significantly reducing development and maintenance costs.

| Component | Technology | Purpose |
|---|---|---|
| **UI Framework** | Flutter 3.x / Dart | Cross-platform mobile app for iOS and Android |
| **State Management** | BLoC (Business Logic Component) | Separates UI logic from business rules; predictable state |
| **HTTP Client** | Dio | Handles REST API calls, interceptors, error handling, retries |

| Component | Technology | Purpose |
|---|---|---|
| **Authentication** | Flutter Secure Storage + JWT | Secure token storage and session management |
| **Push Notifications** | Firebase Cloud Messaging (FCM) | Real-time alerts for lease events, payments, maintenance |
| **Offline Support** | Hive / SQLite | Caches critical data for offline access |
| **Navigation** | GoRouter | Declarative, deep-link-capable navigation |

## 5.2 .NET Core API Layer

The back-end API is built on ASP.NET Core 8, exposing a RESTful interface consumed by the Flutter app, Oracle Financials, and any future third-party integrations. It is hosted on Azure App Service with auto-scaling enabled.

| Layer | Technology | Responsibility |
|---|---|---|
| **API Gateway** | ASP.NET Core 8 Minimal APIs | Route management, rate limiting, response caching, versioning |
| **Authentication** | ASP.NET Core Identity + JWT Bearer | Token issuance, validation, and refresh |
| **Business Logic** | Domain Services / CQRS Pattern | Enforce business rules, orchestrate use cases |
| **Data Access** | Entity Framework Core 8 + Dapper | ORM for complex queries; Dapper for raw Oracle SQL performance |
| **Caching** | Redis (Azure Cache for Redis) | Distributed cache for sessions, lookup data, API responses |
| **Logging** | Serilog + Azure Application Insights | Structured logging, distributed tracing, and alerting |
| **Background Jobs** | Hangfire | Scheduled tasks: report generation, sync jobs, notifications |
| **API Documentation** | Swagger / OpenAPI 3.0 | Auto-generated API documentation for developers and integrators |

## 5.3 Application Services Catalogue

| Service | Responsibility | Integration Points |
|---|---|---|
| **Property Service** | CRUD operations for property records, units, and amenities | Oracle DB, Document Storage |
| **Lease Service** | Manage lease lifecycle, generate contracts, track renewals | Oracle DB, Oracle AP/AR |
| **Financial Service** | Invoice generation, payment processing, GL reconciliation | Oracle DB, Oracle GL/AP/AR |
| **Maintenance Service** | Work order management, vendor assignment, tracking | Oracle DB, Notification Service |
| **Reporting Service** | Aggregated dashboards, export to PDF/Excel | Oracle DB, Azure Blob Storage |
| **Notification Service** | Email, SMS, and push notification delivery | FCM, SendGrid, Azure Service Bus |
| **Auth Service** | JWT token management, MFA, role-based access control | Azure AD, Oracle DB |
| **Audit Service** | Immutable event log for all state changes | Oracle DB, SIEM |

# 6. Technology Architecture

The Technology Architecture layer describes the infrastructure, system software, and physical nodes that support the application and business layers. This corresponds to the ArchiMate Technology Layer. IProperty is designed as a hybrid deployment: the application tier runs on Azure Cloud while the Oracle 11g database tier is hosted on-premise (or in a dedicated VM), with Oracle Data Guard providing high availability.

## 6.1 Infrastructure Components

| Component | Type | Technology | Purpose |
|---|---|---|---|
| **Azure App Service** | Cloud PaaS | Microsoft Azure | Hosts .NET Core API, auto-scales based on load |
| **Azure API Management** | Cloud PaaS | Azure APIM | API gateway, throttling, developer portal, analytics |
| **Azure Cache for Redis** | Cloud PaaS | Redis 7.x | Distributed caching for session and lookup data |
| **Azure Blob Storage** | Cloud PaaS | Azure Storage | Document storage for lease contracts, reports, media |
| **Azure Service Bus** | Cloud PaaS | Azure Messaging | Asynchronous message queue for event-driven services |
| **Azure Application Insights** | Cloud PaaS | Azure Monitor | Distributed tracing, performance monitoring, alerting |
| **Oracle 11g DB Server** | On-Premise / VM | Oracle 11g R2 | Primary relational database for all IProperty data |
| **Oracle Data Guard** | On-Premise / VM | Oracle DG 11g | Standby database for disaster recovery and read offload |
| **RMAN Backup** | On-Premise / VM | Oracle RMAN | Scheduled full and incremental backups |
| **Load Balancer** | Cloud | Azure Load Balancer | Distributes traffic across API instances |
| **CDN** | Cloud | Azure CDN | Static asset delivery for the Flutter web build |
| **VPN Gateway** | Hybrid | Azure VPN Gateway | Secure tunnel between Azure and on-premise Oracle DB |

## 6.2 Oracle 11g Database Architecture

Oracle Database 11g Release 2 is the persistence backbone of IProperty. The database is structured to support high transaction volumes, complex queries for reporting, and seamless integration with Oracle Financials.

- Primary Instance: Hosts all IProperty schemas including PROP_MGMT, LEASE_MGMT, FIN_INTEGRATION, and AUDIT_LOG
- Data Guard Standby: Physical standby database on a separate server, synchronised in real-time via redo log shipping
- Tablespace Strategy: Separate tablespaces for data (PROP_DATA_TS), indexes (PROP_IDX_TS), and LOBs (PROP_LOB_TS)
- Connection Pooling: Oracle Universal Connection Pool (UCP) managed by the .NET Core API layer
- PL/SQL Procedures: Complex business logic and batch processing encapsulated in stored procedures
- Oracle Scheduler: Manages background jobs including financial reconciliation, report generation, and data archiving

## 6.3 Network Topology

| Network Segment | Components | Security Controls |
|---|---|---|
| **Public Internet** | End users, mobile devices | HTTPS/TLS 1.3 only, Azure DDoS Protection |
| **DMZ** | Azure API Management, Load Balancer, WAF | Web Application Firewall, IP whitelisting |
| **Application Tier** | Azure App Service, Redis, Service Bus | Azure Private Endpoints, NSG rules |
| **Data Tier** | Oracle 11g (on-premise/VM) | VPN Tunnel, TDE encryption, Oracle Audit Vault |
| **Integration Tier** | Oracle Financials Cloud | OAuth 2.0, IP whitelisting, TLS mutual auth |

# 7. Integration Architecture

Integration is a core architectural concern for IProperty. The platform must maintain real-time and near-real-time data synchronisation with Oracle Financials Cloud, ensuring that financial transactions originating in IProperty (rent invoices, payment receipts, expense vouchers) are accurately reflected in the Oracle GL, AP, AR, FA, and HR modules.

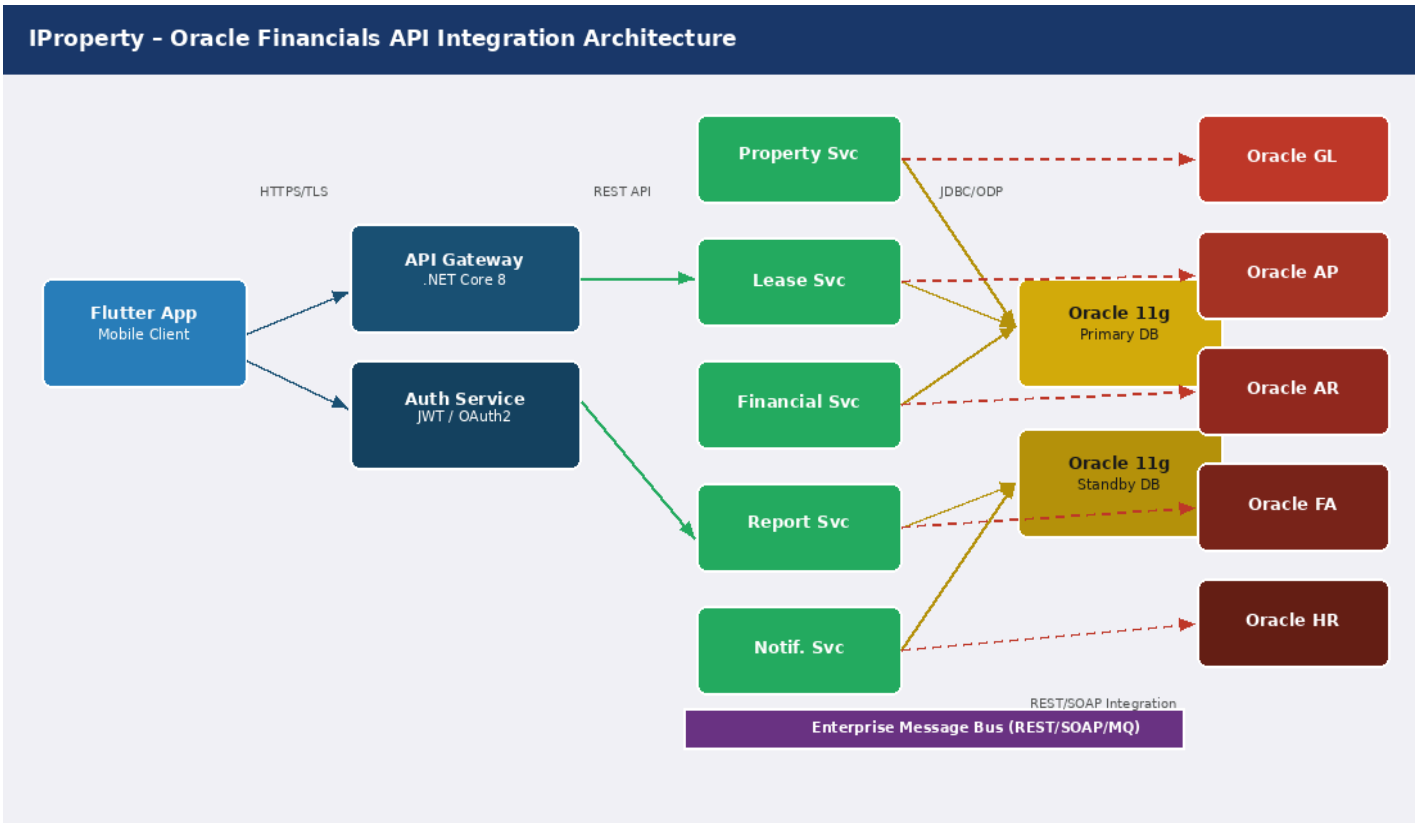


Figure 3 – IProperty Oracle Financials API Integration Architecture

## 7.1 Integration Patterns

| Pattern | Use Case | Technology |
|---|---|---|
| **Synchronous REST** | Real-time data lookup (property details, tenant info) | HTTP/HTTPS REST, JSON payloads |
| **Asynchronous Messaging** | Event-driven financial posting (invoices, payments) | Azure Service Bus, Oracle Financials webhooks |
| **Batch Synchronisation** | End-of-day GL reconciliation and data archiving | Hangfire scheduled jobs, SFTP file transfer |
| **Event Streaming** | Audit trail and change data capture | Azure Event Hub, Oracle LogMiner |
| **Webhook** | Oracle Financials notifying IProperty of approvals | HTTPS POST, HMAC signature validation |

## 7.2 Oracle Financials Module Integration

| Oracle Module | Integration Direction | Data Exchanged | Frequency |
|---|---|---|---|
| **General Ledger (GL)** | IProperty → Oracle GL | Journal entries for rent revenue, expenses, provisions | Daily batch + real-time for critical entries |
| **Accounts Payable (AP)** | IProperty → Oracle AP | Vendor invoices for maintenance, utilities, contractors | Real-time on invoice creation |
| **Accounts Receivable (AR)** | Oracle AR → IProperty | Payment receipts, AR aging, collection status | Real-time via webhook |
| **Fixed Assets (FA)** | IProperty → Oracle FA | Asset acquisition, depreciation, disposal | Monthly batch |
| **Human Resources (HR)** | Oracle HR → IProperty | Employee data for system access provisioning | Daily sync |

## 7.3 API Gateway & Integration Middleware

The ASP.NET Core API Gateway serves as the central integration orchestrator. It handles protocol translation (REST to SOAP where required by Oracle Financials legacy interfaces), payload transformation (JSON to Oracle's XML format), authentication relay (JWT to OAuth 2.0), and error handling with retry policies.

- Protocol Support: REST (primary), SOAP (Oracle legacy modules), SFTP (batch file transfer)
- Payload Transformation: AutoMapper and custom serialisers handle JSON ↔ Oracle XML conversion
- Authentication: OAuth 2.0 client credentials flow for Oracle Financials; JWT Bearer for internal services
- Error Handling: Polly resilience library provides exponential backoff, circuit breakers, and fallback policies
- Idempotency: All integration endpoints are idempotent to prevent duplicate financial postings
- Monitoring: All integration calls are logged to Azure Application Insights with correlation IDs

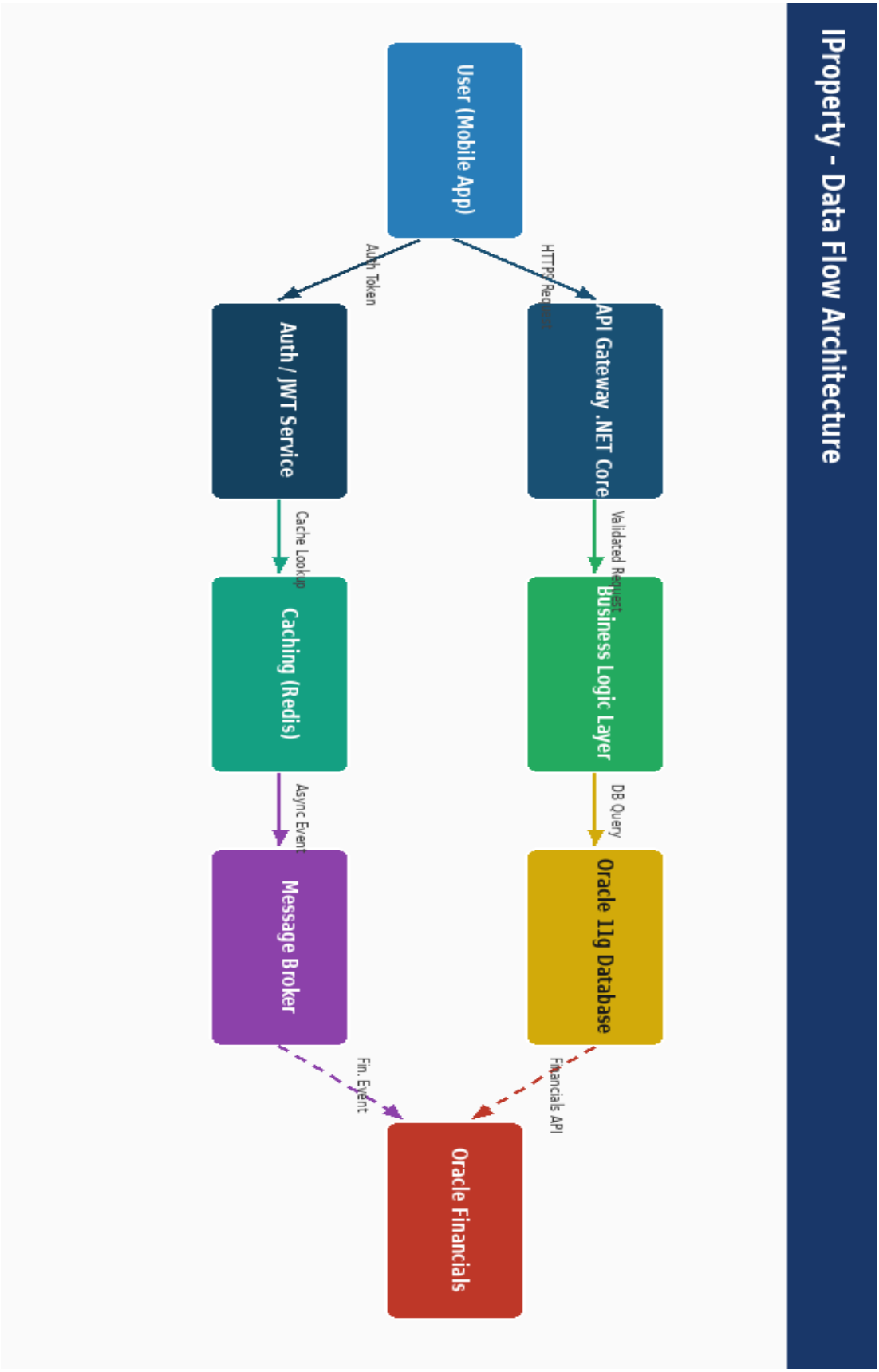


Figure 4 – IProperty Data Flow Architecture

# 8. Data Architecture

The Data Architecture defines how data is structured, stored, governed, and flows across the IProperty platform. Oracle 11g serves as the system of record, with well-defined schemas, data models, and data governance policies to ensure consistency, integrity, and auditability.

## 8.1 Data Domains

| Data Domain | Schema | Key Entities | Oracle Tablespace |
|---|---|---|---|
| **Property Management** | PROP_MGMT | Properties, Units, Amenities, Documents, Valuations | PROP_DATA_TS |
| **Lease Management** | LEASE_MGMT | Leases, Tenants, Clauses, Renewals, Terminations | PROP_DATA_TS |
| **Financial Data** | FIN_MGMT | Invoices, Payments, Journal Entries, Budgets, Costs | FIN_DATA_TS |
| **Maintenance** | MAINT_MGMT | Work Orders, Vendors, Assets, SLA Tracking | PROP_DATA_TS |
| **Integration** | FIN_INTEGRATION | Oracle Financials Sync Log, API Event Queue, Mapping Tables | INT_DATA_TS |
| **Security & Audit** | AUDIT_LOG | Access Logs, Change Events, API Audit Trail | AUD_DATA_TS |

## 8.2 Data Governance

IProperty implements a data governance framework to ensure data quality, lineage, and compliance:

- Data Ownership: Each data domain has a designated data owner responsible for quality and compliance
- Data Classification: All data fields are classified as Public, Internal, Confidential, or Restricted
- Retention Policies: Financial records retained for 7 years; audit logs for 5 years; property data retained for asset lifetime
- Data Masking: Sensitive tenant PII is masked in non-production environments
- Change Data Capture: Oracle LogMiner tracks all DML changes for audit and replication purposes
- Master Data Management: Property and tenant master data is governed through a dedicated MDM process

## 8.3 Database Performance Design

| Strategy | Implementation | Benefit |
|---|---|---|
| **Partitioning** | Range partitioning on date columns (invoices, audit logs) | Improved query performance on large time-series tables |
| **Indexing** | B-tree indexes on FK columns; function-based for case-insensitive search | Fast lookups and join performance |
| **Materialized Views** | Pre-aggregated reporting views refreshed on schedule | Sub-second dashboard query response |
| **Connection Pooling** | Oracle UCP with min/max pool configuration in .NET Core | Reduces connection overhead; handles spikes |
| **Query Optimisation** | SQL execution plan reviews; hints where needed | Prevents full table scans on critical queries |

# 9. Security Architecture

Security is a foundational architecture concern for IProperty, not an afterthought. The security architecture follows a defence-in-depth approach, applying controls at every layer: perimeter, network, application, data, and integration. The model aligns to the ISO 27001 information security framework and the OWASP API Security Top 10.

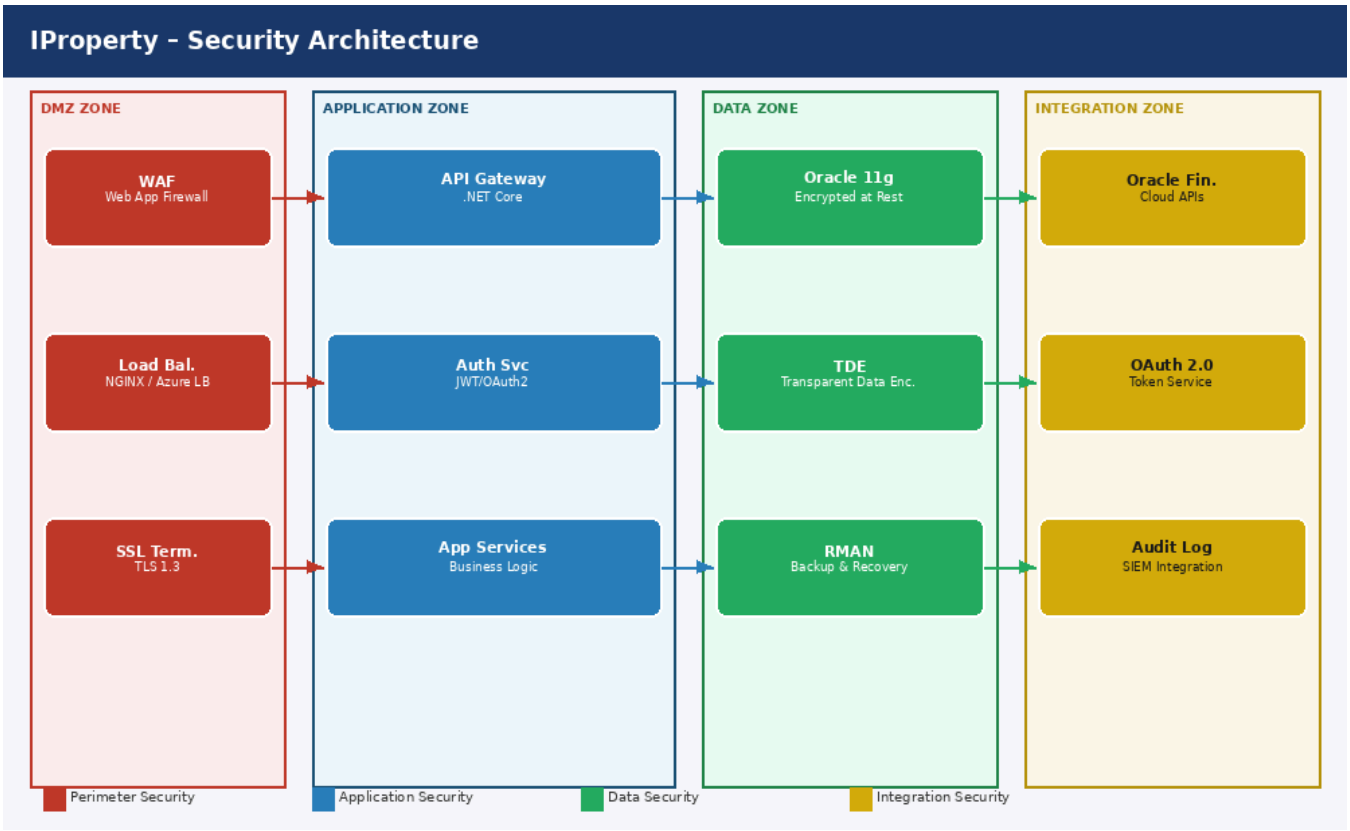


Figure 5 – IProperty Security Architecture (Defence in Depth)

## 9.1 Security Controls by Layer

| Layer | Control | Technology |
|---|---|---|
| **Perimeter** | Web Application Firewall (WAF) | Azure WAF (OWASP ruleset 3.2) |
| **Perimeter** | DDoS Protection | Azure DDoS Protection Standard |
| **Perimeter** | TLS Termination | TLS 1.3 only; TLS 1.0/1.1 disabled |
| **Network** | Network Security Groups | Azure NSG with least-privilege inbound/outbound rules |
| **Network** | Private Endpoints | Azure Private Link for all PaaS services |

| Layer | Control | Technology |
|---|---|---|
| **Network** | VPN Tunnel | IPSec VPN between Azure and on-premise Oracle DB |
| **Application** | JWT Authentication | RSA-signed JWTs with 15-minute expiry |
| **Application** | Role-Based Access Control | Fine-grained RBAC with least-privilege roles |
| **Application** | Multi-Factor Authentication | Azure AD MFA for all administrative accounts |
| **Application** | API Rate Limiting | Azure APIM throttling policies |
| **Data** | Transparent Data Encryption (TDE) | Oracle Advanced Security TDE for all tablespaces |
| **Data** | Column-Level Encryption | AES-256 for PII fields (NIC, bank account, contact) |
| **Data** | Database Audit | Oracle Unified Auditing for all DDL/DML operations |
| **Integration** | OAuth 2.0 | Client credentials flow for Oracle Financials API calls |
| **Integration** | IP Whitelisting | Oracle Financials API restricted to IProperty IP ranges |
| **Integration** | Payload Signing | HMAC-SHA256 signatures on all integration payloads |

## 9.2 Identity & Access Management

IProperty implements a centralised Identity and Access Management (IAM) model using Azure Active Directory as the identity provider. The following roles are defined:

| Role | Access Level | Description |
|---|---|---|
| **System Administrator** | Full | All modules, user management, system configuration |
| **Finance Manager** | Finance + Reporting | Financial operations, GL reconciliation, reporting |
| **Property Manager** | Property + Lease | Property records, lease management, maintenance |
| **Finance Officer** | Finance (Read/Write) | Invoice processing, payment recording, AP/AR |
| **Property Officer** | Property + Lease (Read/Write) | Day-to-day property and lease operations |
| **Read-Only Analyst** | All Modules (Read) | Reporting and analytics only |
| **Tenant (External)** | Tenant Portal Only | Own lease, payments, maintenance requests |

| Role | Access Level | Description |
|---|---|---|
| **Oracle Financials** | Integration Only | API-level access via OAuth 2.0 service account |

# 10. Deployment Architecture

The IProperty deployment architecture adopts a hybrid model: the application layer is deployed on Microsoft Azure cloud infrastructure, while the Oracle 11g database tier resides on-premise (or on a dedicated Azure VM) connected via a secure VPN tunnel. This approach provides cloud elasticity for the application layer while preserving data sovereignty for financial data on Oracle.

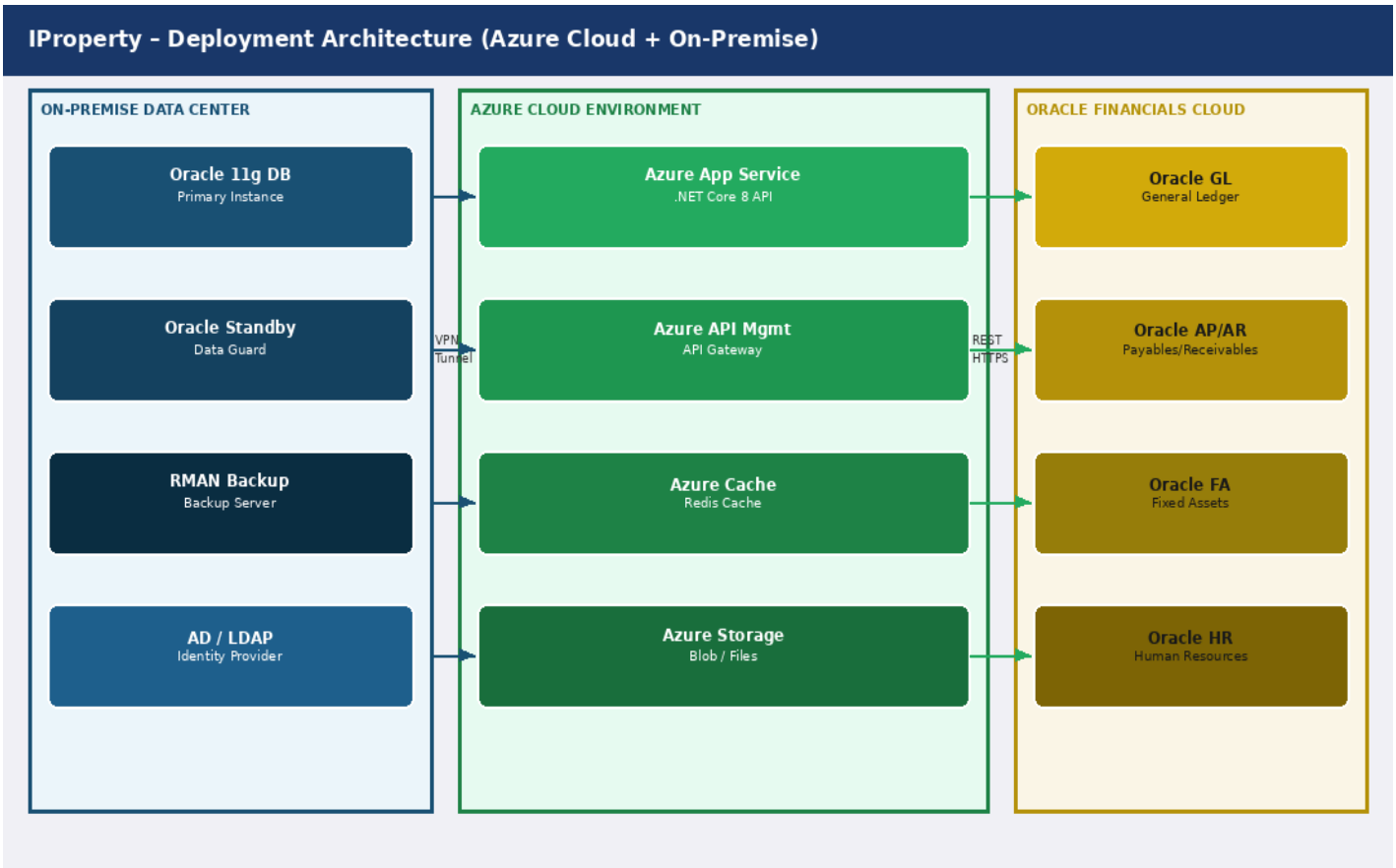


Figure 6 – IProperty Deployment Architecture (Azure + On-Premise)

## 10.1 Environment Strategy

| Environment | Purpose | Infrastructure | Data Policy |
|---|---|---|---|
| **Development** | Feature development and unit testing | Azure Dev (single instance, shared) | Anonymised seed data |
| **Testing / QA** | Integration and regression testing | Azure Test (multi-instance) | Anonymised copy of production |
| **Staging / UAT** | User acceptance testing and pre-release validation | Azure Staging (production-like) | Masked production data |
| **Production** | Live system serving real users | Azure Production (auto-scaled, HA) | Full production data, encrypted |

## 10.2 Azure Resource Configuration

| Azure Resource | SKU / Configuration | Purpose |
|---|---|---|
| **App Service Plan** | P2v3 (2 vCPU, 8GB RAM) – auto-scale to 10 instances | Host .NET Core 8 API |
| **Azure API Management** | Developer/Standard tier | API gateway, throttling, developer portal |
| **Azure Cache for Redis** | Standard C2 (6GB) | Distributed cache |
| **Azure Service Bus** | Standard tier, 3 queues, 2 topics | Async messaging |
| **Azure Blob Storage** | LRS, Hot tier | Documents, reports, media |
| **Azure Application Insights** | Pay-as-you-go | Monitoring and tracing |
| **Azure VPN Gateway** | VpnGw1 (650Mbps) | Secure tunnel to on-premise Oracle |
| **Azure Load Balancer** | Standard | Traffic distribution across API instances |

## 10.3 CI/CD Pipeline

IProperty employs Azure DevOps for continuous integration and deployment, with the following pipeline stages:

1. Code commit to Azure DevOps Git repository triggers the pipeline
2. Build stage: dotnet build, flutter build, unit tests
3. Static analysis: SonarQube code quality and security scanning
4. Artifact creation: Docker image for .NET Core API; APK/IPA for Flutter
5. Deploy to Development environment automatically on merge to develop branch
6. Deploy to Testing environment after QA approval gate
7. Deploy to Staging after UAT sign-off
8. Deploy to Production after Change Advisory Board (CAB) approval

# 11. Performance & Scalability Architecture

IProperty is designed to support a growing property portfolio and user base without architectural changes. The scalability strategy employs horizontal scaling of the application tier, vertical optimisation of the Oracle database, and intelligent caching to reduce database load.

## 11.1 Performance Targets

| Metric | Target | Measurement Method |
|---|---|---|
| **API Response Time (p95)** | < 400ms | Azure Application Insights percentile tracking |
| **API Response Time (p99)** | < 1,000ms | Azure Application Insights percentile tracking |
| **Database Query Time (p95)** | < 200ms | Oracle AWR reports |
| **Flutter App Startup Time** | < 3 seconds (cold start) | Firebase Performance Monitoring |
| **Oracle Financials Sync Latency** | < 5 seconds (real-time) | Custom integration dashboard |
| **System Availability** | 99.9% (excluding planned maintenance) | Azure Monitor uptime checks |
| **Concurrent Users** | 500 simultaneous users (initial), scalable to 5,000 | Azure Load Testing |

## 11.2 Caching Strategy

| Cache Layer | Data Cached | TTL | Technology |
|---|---|---|---|
| **API Response Cache** | GET endpoint responses (property list, lookup data) | 5 minutes | Azure API Management + Redis |
| **Session Cache** | User session, JWT token metadata | 15 minutes | Redis |
| **Reference Data Cache** | Property types, lease templates, currency codes | 1 hour | Redis |
| **Reporting Cache** | Pre-aggregated dashboard metrics | 30 minutes | Redis + Materialized Views |
| **CDN Cache** | Flutter web build static assets | 24 hours | Azure CDN |

## 11.3 Auto-Scaling Configuration

The Azure App Service hosting the .NET Core API is configured with the following auto-scaling rules:

- Scale Out: Add 1 instance when average CPU > 70% for 5 minutes
- Scale Out: Add 1 instance when average memory > 80% for 5 minutes
- Scale In: Remove 1 instance when average CPU < 30% for 15 minutes
- Minimum Instances: 2 (high availability during low traffic)
- Maximum Instances: 10 (cost ceiling during peak periods)
- Scheduled Scaling: Pre-scale to 4 instances during business hours (8AM–8PM)

# 12. Reliability & Availability Architecture

IProperty is a business-critical system managing financial and contractual data. A comprehensive reliability and availability strategy is therefore mandatory. The architecture targets a 99.9% uptime SLA, achieved through redundant infrastructure, automated failover, and rigorous backup and recovery procedures.

## 12.1 High Availability Design

| Component | HA Strategy | Recovery Time Objective (RTO) | Recovery Point Objective (RPO) |
|---|---|---|---|
| **.NET Core API** | Azure App Service with 2+ instances; load balancer | < 1 minute (auto-recovery) | 0 (stateless) |
| **Redis Cache** | Azure Cache for Redis with geo-replication | < 2 minutes | < 1 minute |
| **Oracle 11g (Primary)** | Oracle Data Guard with physical standby | < 10 minutes (manual failover) | < 5 seconds (redo log gap) |
| **Oracle Standby** | Automatic failover via Oracle Data Guard Broker | < 5 minutes | 0 (sync replication) |
| **Azure Service Bus** | Premium tier with geo-disaster recovery | < 5 minutes | < 1 minute |
| **Oracle Financials** | Managed by Oracle (99.5% SLA) | Per Oracle SLA | Per Oracle SLA |

## 12.2 Backup & Recovery

- Oracle RMAN: Full backup weekly (Sunday 02:00 AM); incremental backup nightly (02:00 AM)
- Backup Retention: 35 days on-site; 90 days off-site (Azure Blob Storage, geo-redundant)
- Backup Encryption: AES-256 encryption of all RMAN backup sets
- Restore Testing: Monthly full restore drill to validate backup integrity
- Application State: .NET Core is stateless; Redis data reconstructed from Oracle on restart
- Document Storage: Azure Blob Storage with LRS + monthly geo-redundant backup

## 12.3 Disaster Recovery Plan

| Disaster Scenario | Detection | Response | Expected RTO |
|---|---|---|---|
| **API layer failure** | Azure Monitor health probe failure | Auto-restart, scale to new instance | < 2 minutes |
| **Oracle Primary DB failure** | Oracle DG Broker alert | Activate standby database (manual approval) | < 10 minutes |

| Disaster Scenario | Detection | Response | Expected RTO |
|---|---|---|---|
| **Azure region outage** | Azure Service Health alert | Redirect DNS to secondary region | < 30 minutes |
| **Oracle Financials outage** | Integration health check failure | Queue all outbound events; replay on recovery | 0 downtime (queued) |
| **Ransomware / data corruption** | Security alert or anomaly detection | Isolate, restore from last known good RMAN backup | < 2 hours |

# 13. Operational Architecture

The Operational Architecture defines how IProperty is monitored, maintained, and operated in production. It covers the observability stack, incident management, change management, and support model.

## 13.1 Monitoring & Observability

| Observability Pillar | Tool | What It Monitors |
|---|---|---|
| **Application Performance Monitoring** | Azure Application Insights | API latency, error rates, dependency calls, user flows |
| **Infrastructure Monitoring** | Azure Monitor | CPU, memory, disk, network across all Azure resources |
| **Database Monitoring** | Oracle Enterprise Manager | Oracle DB performance, wait events, top SQL, tablespace usage |
| **Log Management** | Azure Log Analytics | Centralised log aggregation from all application and infra components |
| **Distributed Tracing** | Azure Application Insights (W3C TraceContext) | End-to-end request tracing from Flutter app to Oracle DB |
| **Security Monitoring** | Microsoft Sentinel (SIEM) | Security events, anomaly detection, Oracle audit logs |
| **Integration Monitoring** | Custom dashboard (Power BI) | Oracle Financials sync status, queue depths, error rates |
| **Uptime Monitoring** | Azure Monitor Availability Tests | External HTTP probes every 5 minutes from multiple regions |

## 13.2 Alerting & Incident Management

Alerts are defined for all critical thresholds and are routed to the operations team via Azure Monitor Action Groups:

- P1 (Critical): System unavailable, Oracle DB down, financial sync failure – immediate page to on-call engineer
- P2 (High): API error rate > 5%, DB response > 2s, queue depth > 1,000 – alert within 15 minutes
- P3 (Medium): Memory > 85%, disk > 80%, non-critical service degradation – business hours alert
- P4 (Low): Non-critical warnings, informational alerts – daily digest email

## 13.3 Change Management

All changes to the IProperty production environment follow a formal change management process:

9. Change Request raised in Azure DevOps Work Items with impact and rollback plan
10. Technical review by architecture team
11. Change Advisory Board (CAB) approval for production changes
12. Deploy to staging environment and validate
13. Production deployment via CI/CD pipeline with approval gate
14. Post-deployment validation and monitoring for 24 hours
15. Change closure with lessons learned if applicable

# 14. Risk & Compliance Architecture

The Risk and Compliance Architecture identifies the key architectural risks for IProperty, the likelihood and impact of each risk, and the mitigating controls that have been built into the architecture.

## 14.1 Architecture Risk Register

| Risk ID | Risk Description | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| **R01** | Oracle 11g end-of-extended-support (2024) creates vendor support risk | High | High | Architectural roadmap includes Oracle 19c migration in Phase 2; current Oracle Premier Support extended contract in place |
| **R02** | Oracle Financials API changes break integration | Medium | High | Version-pinned API contracts; integration smoke tests in CI/CD pipeline; change notification agreement with Oracle |
| **R03** | Flutter app security vulnerability in third-party packages | Medium | High | Regular dependency audits (OWASP Dependency-Check); Flutter package pinning; code signing |
| **R04** | Azure cloud region outage | Low | High | Multi-region deployment readiness; Azure Site Recovery configured; RTO < 30 minutes |
| **R05** | Oracle DB performance degradation under load | Medium | High | Oracle AWR-based tuning; Data Guard offloads reporting queries; Redis caching reduces DB load |
| **R06** | Data breach of tenant PII | Low | Critical | TDE encryption, column-level AES-256, Oracle Audit Vault, |

| Risk ID | Risk Description | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| | | | | Azure Sentinel SIEM, RBAC |
| **R07** | Integration data mismatch causing financial errors | Medium | Critical | Idempotent APIs, reconciliation jobs, dual-entry validation, audit trail for all financial postings |

## 14.2 Compliance Framework

IProperty is designed to support compliance with the following regulatory and standards frameworks:

| Framework | Applicability | Architectural Controls |
|---|---|---|
| **ISO 27001** | Information Security Management | TDE, access control, audit logs, SIEM, incident management |
| **POPIA (South Africa)** | Personal Information Protection | PII encryption, data masking, retention policies, consent management |
| **IFRS 16 (Lease Accounting)** | Financial Reporting | Oracle AR/GL integration, lease liability calculations, right-of-use asset tracking |
| **OWASP API Security Top 10** | Application Security | Input validation, rate limiting, authentication, authorisation, logging |
| **Oracle Financials Controls** | Financial Integrity | Dual-entry validation, GL reconciliation, audit trail, segregation of duties |

# 15. Architecture Roadmap

The IProperty architecture is designed with an evolutionary roadmap in mind. The following phases outline the planned architectural progression from the initial release through to a fully mature, cloud-native platform.

| Phase | Timeline | Key Deliverables | Architecture Changes |
|---|---|---|---|
| **Phase 1 – Foundation** | Q2 2026 | Flutter app, .NET Core API, Oracle 11g, Oracle Financials integration | Initial architecture as documented in this SAD |
| **Phase 2 – Enhancement** | Q4 2026 | Advanced reporting, offline-first Flutter app, Oracle 19c migration | Oracle DB upgrade; enhanced caching; Power BI embedded |
| **Phase 3 – Scale** | Q2 2027 | Multi-tenancy, marketplace integrations, AI-powered analytics | Microservices decomposition; event streaming with Azure Event Hub |
| **Phase 4 – Autonomy** | Q4 2027 | Predictive maintenance, automated lease renewals, AI valuation engine | Azure AI/ML services; LLM integration for document processing |

## 15.1 Technical Debt Items

| Item | Description | Priority | Target Phase |
|---|---|---|---|
| **Oracle 11g Upgrade** | Migrate to Oracle 19c for extended support and performance | High | Phase 2 |
| **Microservices Decomposition** | Decompose monolithic .NET Core API into bounded-context services | Medium | Phase 3 |
| **GraphQL API** | Add GraphQL alongside REST for more efficient Flutter data fetching | Low | Phase 2 |
| **Zero-Trust Network** | Full zero-trust implementation replacing VPN tunnel | Medium | Phase 2 |
| **Database as a Service** | Migrate Oracle to Azure Database for Oracle (when GA) | Low | Phase 4 |

# Appendix A – Glossary of Terms

| Term | Definition |
|---|---|
| **ArchiMate** | The Open Group Architecture Modelling Language (version 3.1); a vendor-neutral EA notation |
| **API Gateway** | A server that acts as the entry point for API clients, handling routing, auth, and rate limiting |
| **BLoC** | Business Logic Component; a Flutter state management pattern |
| **CDN** | Content Delivery Network; distributes static assets close to end users |
| **CQRS** | Command Query Responsibility Segregation; separates read and write models |
| **Data Guard** | Oracle's high-availability solution providing physical standby databases |
| **DXA** | DocX Units; 1440 DXA = 1 inch (used in OOXML document formats) |
| **GL** | General Ledger; the central repository of accounting data in Oracle Financials |
| **JWT** | JSON Web Token; a compact, URL-safe means of representing claims between two parties |
| **RMAN** | Recovery Manager; Oracle's backup and recovery tool |
| **TDE** | Transparent Data Encryption; Oracle feature encrypting data files without application changes |
| **VPN** | Virtual Private Network; encrypted tunnel between two network endpoints |
| **WAF** | Web Application Firewall; filters and monitors HTTP traffic to protect against attacks |

# Appendix B – Reference Documents

| Document | Reference |
|---|---|
| **ArchiMate 3.1 Specification** | The Open Group – https://pubs.opengroup.org/architecture/archimate3-doc/ |
| **Oracle Database 11g Documentation** | Oracle – https://docs.oracle.com/cd/B28359_01/ |
| **ASP.NET Core 8 Documentation** | Microsoft – https://learn.microsoft.com/en-us/aspnet/core/ |
| **Flutter SDK Documentation** | Google – https://docs.flutter.dev/ |

| Document | Reference |
|---|---|
| **Oracle Financials Cloud API Guide** | Oracle – https://docs.oracle.com/en/cloud/saas/financials/ |
| **Azure Architecture Center** | Microsoft – https://learn.microsoft.com/en-us/azure/architecture/ |
| **OWASP API Security Top 10** | OWASP Foundation – https://owasp.org/www-project-api-security/ |
| **ISO/IEC 27001:2022** | ISO – Information Security Management Standard |

# Appendix C – Document Approval

| Role | Name | Signature | Date |
|---|---|---|---|
| **Enterprise Architect** | | | |
| **Solution Architect** | | | |
| **Head of IT** | | | |
| **Finance Director** | | | |
| **Project Manager** | | | |
| **Security Officer** | | | |